

Constructor with program in C++

In C++, a constructor is a special member function of a class that is automatically called when an object of that class is created. It is used to initialize the object. A constructor has the same name as the class and does not have a return type.

- **Constructor** ki madad se jab object banaya jaata hai, uske saare members ko initial values mil jaati hain bina kisi extra step ke.
- Agar aap setter functions ka use karte hain, toh आपको manually har object ka initialization karna padta hai, jo bad mein galtiyan karne ka chance badha deta hai.

```
1 #include<iostream>
2 using namespace std;
3 class Rectangle {
4 private:
5     int width, height;
6 public:
7     // Default constructor
8     Rectangle() {
9         width = 0;
10        height = 0;}
11    // Parameterized constructor
12    Rectangle(int w, int h) {
13        width = w;
14        height = h;}
15
16    void display() {
17        cout << "Width: " << width << ", Height: " << height << endl
18        ;
19    }
20 };
21 int main() {
22     Rectangle rect1; // Calls default constructor
23     Rectangle rect2(5, 10); // Calls parameterized constructor
24     rect1.display();
25     rect2.display();
26 }
```

Width: 0, Height: 0
Width: 5, Height: 10
=== Code Execution Suc

Overriding - Overloaded – Overloading

→

Overloading (Function Overloading) (overloaded function)

main.cpp

Share

Run

Output

```

1 #include <iostream>
2 using namespace std;
3
4 class Calculator {
5 public:
6     // Function to add two integers
7     int add(int a, int b) {
8         return a + b;    }
9     // Function to add three integers (overloaded function)
10    int add(int a, int b, int c) {
11        return a + b + c;    }
12    // Function to add two double values (overloaded function)
13    double add(double a, double b) {
14        return a + b;    }
15 };
16 int main() {
17     Calculator calc;
18
19     // Calling overloaded functions
20     cout << "Sum of 2 and 3: " << calc.add(2, 3) << endl;
21     cout << "Sum of 1, 2, and 3: " << calc.add(1, 2, 3) << endl;
22     cout << "Sum of 2.5 and 3.7: " << calc.add(2.5, 3.7) << endl;
23     return 0;
24 }
25

```

Sum of 2 and 3: 5
Sum of 1, 2, and 3: 6
Sum of 2.5 and 3.7: 6.2

=== Code Execution Success

Feature	Function Overloading	Method Overriding
Definition	Ek hi function naam ke saath multiple functions create karna, jisme parameters alag ho.	Parent class ke method ko child class mein redefine karna.
Purpose	Ek function ko multiple ways mein use karne ke liye.	Parent class ke method ka specific behavior child class mein dena.
Function Signature	Overloaded functions mein signature (function name + parameter types) alag hote hain.	Overridden method ka signature parent class ke method ke signature ke saath bilkul same hona chahiye.
Usage	Ek hi function ko alag data types ya number of parameters ke saath call karne ke liye.	Inheritance ka use karte hue parent class ka function child class mein modify karne ke liye.
Compile-time / Run-time	Compile-time polymorphism (Function Resolution during compilation)	Run-time polymorphism (Function resolution during execution)

main.cpp	Run	Output
<pre>1 #include <iostream> 2 using namespace std; 3 class Animal { 4 public: 5 // Virtual function 6 virtual void sound() { 7 cout << "Animals make sounds!" << endl; 8 } 9 }; 10 class Dog : public Animal { 11 public: 12 void sound() override { // Override in Dog class 13 cout << "Dog barks!" << endl; 14 } 15 }; 16 int main() { 17 Animal* animal; 18 Dog dog; 19 20 animal = &dog; 21 animal->sound(); // Calls Dog's sound() function 22 23 return 0; 24 }</pre>		Dog barks!

Virtual Function:

Virtual Function ek function hota hai jo **parent class** mein declare kiya jaata hai aur jab yeh function **child class** mein override hota hai, tab runtime pe decide hota hai ki kis class ka function call hoga (yani, child class ya parent class ka).

Virtual function ka use C++ mein run-time polymorphism achieve karne ke liye hota hai. Iska matlab yeh hai ki aap **base class pointer** ya **reference** ka use karte hue **derived class** ke function ko call kar sakte hain.

Structure and Struct

Structure aur **struct** dono ka concept ek hi hai, lekin C++ mein in dono ka use kaafi similar hota hai. **struct** C++ mein ek keyword hai jo structure define karne ke liye use hota hai, aur **structure** ek data type hota hai. Dono ka basic idea ek hi hai, lekin unka usage aur context thoda different ho sakta hai.

1. Default Access Specifiers:

- C mein structure ke members ka default access **public** hota hai.
- C++ mein bhi structure ke members ka default access **public** hota hai, lekin **class** ka default access **private** hota hai.

Difference between `struct` in C and `struct` in C++

Feature	<code>struct</code> in C	<code>struct</code> in C++
Access Modifier	Default is <code>public</code>	Default is <code>public</code>
Object-Oriented Support	No support for OOP (Object-Oriented Programming)	Supports inheritance, polymorphism, etc. (like classes)
Functions Inside <code>struct</code>	No functions are typically used inside a <code>struct</code>	Can define member functions inside a <code>struct</code>
Data Encapsulation	Limited, no access control mechanisms	Supports data encapsulation and OOP concepts
Use	Grouping related data in a procedural way	Grouping data and using OOP features

```
main.cpp
1 #include <stdio.h>
2
3 // C structure definition
4 struct Person {
5     char name[50]; // public by default in C
6     int age;        // public by default in C
7 };
8
9 int main() {
10     struct Person person1; // Creating structure variable
11     person1.age = 25;      // Accessing members (public by default)
12     printf("Age: %d\n", person1.age);
13     return 0;
14 }
15
```

Output

Age: 25

=== Code

With oops method →

```
main.cpp
1 #include <iostream>
2 using namespace std;
3 struct Student {
4     string name; int age; float marks;
5     void display() {
6         cout << "Name: " << name << endl;
7         cout << "Age: " << age << endl;
8         cout << "Marks: " << marks << endl; }
9 };
10 // Inheriting the Student structure
11 struct GraduateStudent : public Student {
12     string degree;
13     void display() {
14         // Calling the base class display function
15         Student::display();
16         cout << "Degree: " << degree << endl; } };
17 int main() {
18     // Create an object of GraduateStudent
19     GraduateStudent grad1;
20     grad1.name = "John";
21     grad1.age = 25;
22     grad1.marks = 90.5;
23     grad1.degree = "Masters in Computer Science";
24     // Displaying student information
25     grad1.display();
26     return 0; }

```

Output

Name: John
Age: 25
Marks: 90.5
Degree: Masters in Computer Science

=== Code Execution Successful ===

DYNAMIC MEMORY ALLOCATION :

Dynamic memory allocation ka use tab kiya jata hai jab aapko runtime pe memory allocate karni hoti hai, jo size compile time pe decide nahi hota. C++ mein dynamic memory allocation ke liye **new** aur **delete** keywords ka use hota hai.

main.cpp	Output
<pre> 1 #include <iostream> 2 using namespace std; 3 int main() { 4 // Dynamically allocate memory for a single integer 5 int* ptr = new int; // Allocating memory for one integer 6 *ptr = 100; 7 8 // Accessing the value stored in the dynamically allocated memory 9 cout << "The value of ptr is: " << *ptr << endl; 10 11 // Deallocating the memory 12 delete ptr; 13 14 return 0; 15 } </pre>	<pre> The value of ptr is: 100 === Code Execution Successful === </pre>

new int: Yeh memory allocate karta hai jo ek integer store kar sake.

delete Operator:

- **delete** operator dynamically allocated memory ko free karne ke liye use hota hai.
- Jab aap **new** ka use karte hain, tab memory ko **manually** deallocate karna zaroori hota hai.
- **delete** ka use single variable ke liye hota hai aur **delete[]** ka use array ke liye hota hai.

Use of Virtual Keyword & Program

C++ mein **virtual** keyword ka use inheritance aur polymorphism ke concept ko implement karne ke liye hota hai. Jab hum **virtual function** define karte hain, toh C++ ko yeh bata sakte hain ki run-time polymorphism ko support karna hai. **Virtual functions** ka use hum **base class** mein karte hain, jisse **derived class** mein function ko **override** kiya ja sakta hai.

main.cpp	Output
<pre> 1 #include <iostream> 2 using namespace std; 3 class Animal { 4 public: 5 virtual void sound() { 6 cout << "This is an animal sound" << endl; } 7 virtual ~Animal() {} }; 8 class Dog : public Animal { 9 public: 10 void sound() override { 11 cout << "Woof woof!" << endl; } }; 12 class Cat : public Animal { 13 public: 14 void sound() override { 15 cout << "Meow meow!" << endl; } }; 16 int main() { 17 Animal* animalPtr; 18 Dog dog; 19 Cat cat; 20 animalPtr = &dog; 21 animalPtr->sound(); 22 animalPtr = &cat; 23 animalPtr->sound(); 24 return 0; 25 } 26 </pre>	<pre> Woof woof! Meow meow! === Code Execution Successful === </pre>

Exception handling →

C++ mein exception handling ek mechanism hai jo program ko runtime errors (exceptions) se bachane ke liye use hota hai. Jab koi error occur hota hai, toh program crash nahi hota, balki uss error ko handle karke program ko aage continue karne ka moka milta hai.

C++ mein exception handling ko `try`, `throw`, aur `catch` ke through implement kiya jata hai.

C++ mein Exception Handling ke 3 main parts hote hain:

1. **try block:** Yahan pe wo code likhte hain jahan error aa sakta hai.
2. **throw statement:** Agar error aata hai, toh yeh error ko throw (raise) karta hai.
3. **catch block:** Yahan pe hum error ko handle karte hain.

main.cpp	Output
<pre>1 #include <iostream> 2 using namespace std; 3 4 void divide(int a, int b) { 5 if (b == 0) { 6 throw "Division by zero!"; 7 } 8 cout << a / b << endl; 9 } 10 11 int main() { 12 try { 13 divide(10, 0); 14 } catch (const char* msg) { 15 cout << msg << endl; 16 } 17 18 return 0; 19 }</pre>	<pre>Division by zero! === Code Execution</pre>

- I removed the unnecessary variables `x` and `y` and directly called `divide` with the values 10 and 0.

- The `divide` function directly handles the exception check and prints the result if no exception occurs.

- **try block:** Is block mein woh code hota hai jo exception throw kar sakta hai. Aapke case mein, yeh `divide` function ko call karta hai 10 aur 0 ke arguments ke sath. Kyunki 0 se divide karna allowed nahi hai, yeh exception throw karega.
- **throw statement:** `divide` function ke andar, agar `b 0` hai, toh `throw` statement execute hota hai, jo ek exception (is case mein, string "Division by zero!") ko `catch` block ke paas bhej deta hai.

- **catch block:** Yeh block exception ko catch karta hai jo `try` block mein throw hoti hai. Yeh ek parameter (is case mein, `const char* msg`) leta hai, jo exception hai. Yeh block phir us exception ko handle karta hai, jaise error message print karna.

Agar kuch galat hota hai, `catch` block ensure karta hai ki program crash na ho aur gracefully error ko handle kar sake

Heap Memory kya hai?

Heap memory runtime par dynamically allocate hoti hai. Matlab, jab aapko program ke dauraan memory chahiye hoti hai tab aap usse le sakte ho aur jab kaam ho jaye toh free kar sakte ho.

```
1 #include <iostream>
2 using namespace std;
3
4 int main() {
5     int* ptr = new int; // Ek integer ke liye memory allocate kar
                        // rahe hain heap par
6     *ptr = 100; // Us memory mein value 100 rakh rahe hain
7
8     cout << "Value: " << *ptr << endl;
9
10    delete ptr; // Memory ko free kar rahe hain
11
12    return 0;
13 }
```

Value: 100

=== Code Exec

Difference Heap & Main Memory

Speed	Fast hoti hai kyunki automatically handle hoti hai.	Relatively slow kyunki manually handle karni padti hai.
Memory Size	Limited size hoti hai.	Zyada size hoti hai.
	Zyada memory allocate karne par stack overflow error ho sakti hai.	Large amount of memory allocate kar sakte hain.
Usage	Local variables, function parameters, aur return addresses ke liye use hoti hai.	Dynamic memory allocation ke liye use hoti hai, jaise linked lists, trees, aur large objects.

Aspect	Stack Memory	Heap Memory
Lifetime (Duration)	Function scope tak rahti hai.	Manually control kar sakte hain.
	Function end hone par automatically free ho jati hai.	Jab tak explicitly free nahi karte, tab tak rehti hai.
Allocation & Deallocation	Automatically handle hoti hai.	Manually handle karni padti hai using <code>new</code> aur <code>delete</code> .

C++ mein **access modifiers** ka use class members (variables aur functions) ke access ko control karne ke liye kiya jata hai. C++ mein **3 main access modifiers** hote hain:

- **public**
- **private**
- **protected**

1. public Access Modifier

- **Public members** ko class ke **bahar** se **access** kiya ja sakta hai.
- Yeh generally wo functions ya variables hote hain jo users ko interact karne ke liye available hote hain.

main.cpp	   Share	Run	Output
<pre>1 #include <iostream> 2 using namespace std; 3 4 class MyClass { 5 public: 6 int x; // public variable 7 8 void display() { // public function 9 cout << "Value of x: " << x << endl; 10 } 11 }; 12 13 int main() { 14 MyClass obj; 15 obj.x = 10; 16 obj.display(); // Valid: display function public hai 17 return 0; 18 } 19</pre>			Value of x: 10 === Code Execut

2. private Access Modifier

- **Private members** sirf class ke **andar** hi access kiye ja sakte hain.
- Class ke **bahar** se private members ko directly access nahi kiya ja sakta.
- Yeh usually sensitive data ko hide karne aur encapsulation achieve karne ke liye use hota hai.

main.cpp	Run	Output
<pre> 1 #include <iostream> 2 using namespace std; 3 4 class MyClass { 5 private: 6 int x; 7 public: 8 void setX(int val) { // public function to set x 9 x = val; 10 } 11 int getX() { // public function to get x 12 return x; } }; 13 int main() { 14 MyClass obj; 15 obj.setX(10); 16 cout << "Value of x: " << obj.getX() << endl; 17 return 0; 18 } 19 </pre>	Run	Value of x: 10 === Code Execution Successful ===

3. protected Access Modifier

- **Protected members** ko class ke **andar** aur class ke **derived (child) classes** ke **andar** access kiya ja sakta hai.
- **Class ke bahar** se protected members ko directly access nahi kiya ja sakta.
- Yeh **inheritance** ke time pe kaam aata hai, jab aap derived class ke andar parent class ke protected members ko access karna chahte hain.

main.cpp	Run	Output
<pre> 1 #include <iostream> 2 using namespace std; 3 class Base { 4 protected: 5 int x; // protected variable 6 public: 7 void setX(int val) { // public function to set x 8 x = val; 9 } 10 }; 11 class Derived : public Base { 12 public: 13 void show() { 14 cout << "Vaue of x in Derived class: " << x << endl; 15 } 16 }; 17 int main() { 18 Derived obj; 19 obj.setX(10); // Valid: setX public hai 20 obj.show(); // Valid: show function is accessing protected // member x // obj.x = 20; // Invalid: x protected hai, isliye directly // access nahi kar sakte 21 return 0; 22 } 23 24 </pre>	Run	Vaue of x in Derived class: 10 === Code Execution Successful ===

Difference →

Feature	<code>private</code>	<code>protected</code>
Access in same class	Accessible – Class ke andar directly access kiya ja sakta hai.	Accessible – Class ke andar directly access kiya ja sakta hai.
Access in derived class (Inheritance)	Not accessible – Derived class ke andar directly access nahi kiya ja sakta.	Accessible – Derived class ke andar directly access kiya ja sakta hai.
Access outside class	Not accessible – Class ke bahar se directly access nahi kiya ja sakta.	Not accessible – Class ke bahar se directly access nahi kiya ja sakta.
Purpose	Sensitive data ko protect karna.	Derived class ko base class ke members access karne ki permission dena.
Example	<code>private</code> members ko sirf class ke andar use kiya ja sakta hai.	<code>protected</code> members ko derived class ke andar access kiya ja sakta hai.

STATIC KEYWORD and Without Static →

Without :

```

main.cpp
1 #include <iostream>
2 using namespace std;
3 class MyClass {
4 public:
5     int count;
6     MyClass() {
7         count = 0;
8     }
9     void increment() {
10        count++;
11    }
12    void display() {
13        cout << "Count: " << count << endl;
14    }
15 int main() {
16     MyClass obj1;
17     MyClass obj2;
18     obj1.increment();
19     obj1.display(); // Outputs: Count: 1
20
21     obj2.increment();
22     obj2.display(); // Outputs: Count: 1
23     return 0;
24 }

```

Output

```

Count: 1
Count: 1

```

=== Code Execution ===

- `count` non-static hai, isliye har object apni alag copy rakhta hai.
- `obj1` aur `obj2` ke `count` variables alag-alag hain, isliye dono objects ke `increment` aur `display` calls alag-alag results dete hain.

With Static

main.cpp	Output
<pre>1 #include <iostream> 2 using namespace std; 3 class MyClass { 4 public: 5 static int count; 6 MyClass() { 7 count = 0; 8 } 9 void increment() { 10 count++; 11 } 12 void display() { 13 cout << "Count: " << count << endl; } }; 14 // Static variable ko class ke bahar initialize karte hain 15 int MyClass::count = 0; 16 int main() { 17 MyClass obj1; 18 MyClass obj2; 19 obj1.increment(); 20 obj1.display(); // Outputs: Count: 1 21 obj2.increment(); 22 obj2.display(); // Outputs: Count: 2 23 24 return 0; 25 } 26</pre>	<pre>Count: 1 Count: 2 === Code Execution ===</pre>

- `count` static hai, isliye yeh class ke sabhi objects ke beech share hota hai.
- `obj1` aur `obj2` ka `count` variable common hai, isliye `increment` aur `display` calls dono objects ke liye same result dete hain.

Destructor kya hota hai?

Destructor ek special function hota hai jo tab call hota hai jab object destroy hota hai, matlab jab object ka lifecycle khatam hota hai ya object delete hota hai. Destructor ka kaam resources ko release karna hota hai jo object ne use kiye hain, jaise dynamically allocated memory.

Destructor ka naam class ke naam ke aage tilde (~) laga ke banaya jaata hai aur iske koi parameters nahi hote.

```
class MyClass {
public:
    ~MyClass() {
        // Destructor body
    }
};
```

main.cpp	Output
<pre> 1 #include <iostream> 2 using namespace std; 3 4- class MyClass { 5 public: 6- MyClass() { 7 cout << "Constructor called!" << endl; 8 } 9- ~MyClass() { 10 cout << "Destructor called!" << endl; 11 } 12 }; 13- int main() { 14 MyClass obj; // Jab object banta hai, Constructor call hota hai 15 // Destructor tab call hoga jab object ka scope khatam hoga, yahan par end of main() 16 return 0; 17 } 18 </pre>	<pre> Constructor called! Destructor called! === Code Execution Su </pre>

Dynamic Memory Example:

Jab object dynamically memory allocate karta hai (**heap memory**), to destructor memory ko free karne ke liye use hota hai.

main.cpp	Output
<pre> 1 #include <iostream> 2 using namespace std; 3 4- class MyClass { 5 private: 6 int* data; 7 public: 8- MyClass() { 9 data = new int; // Heap memory allocate hoti hai 10 cout << "Constructor called!" << endl; } 11- ~MyClass() { 12 delete data; // Heap memory free hoti hai 13 cout << "Destructor called!" << endl; } 14 }; 15- int main() { 16 MyClass obj; // Jab object banta hai, constructor call hota hai 17 // Jab main function khatam hota hai, destructor call hota hai 18 return 0; 19 } 20 </pre>	<pre> Constructor called! Destructor called! === Code Execution Suc </pre>

Template Argument kya hota hai?

Templates C++ mein ek tareeka hain flexible aur reusable code likhne ka. Template arguments basically woh specific types ya values hain jo aap templates ko dete ho taaki woh unke sath kaam kar sakein.

Types of Template Arguments:

1. **Type Argument:** Yeh data type specify karta hai, jaise `int`, `double`, etc.
2. **Non-Type Argument:** Yeh ek constant value specify karta hai, jaise `int`, `char`, etc.

main.cpp	Output
<pre> 1 #include <iostream> 2 using namespace std; 3 template <typename T> 4 class MyClass { 5 public: 6 T data; 7 MyClass(T data) : data(data) {} 8 9 void display() { 10 cout << "Data: " << data << endl; } 11 }; 12 int main() { 13 MyClass<int> obj1(100); // T yahan 'int' hai 14 MyClass<double> obj2(3.14); // T yahan 'double' hai 15 MyClass<string> obj3("Hello"); // T yahan 'string' hai 16 17 obj1.display(); // Outputs: Data: 100 18 obj2.display(); // Outputs: Data: 3.14 19 obj3.display(); // Outputs: Data: Hello 20 21 return 0; 22 } </pre>	<pre> Data: 100 Data: 3.14 Data: Hello === Code Execut </pre>

NON- type

main.cpp	Output
<pre> 1 #include <iostream> 2 using namespace std; 3 4 template <int N> 5 class MyArray { 6 public: 7 int arr[N]; 8 void fill() { 9 for (int i = 0; i < N; ++i) { 10 arr[i] = i * 10; } 11 } 12 void display() { 13 for (int i = 0; i < N; ++i) { 14 cout << arr[i] << " "; 15 } 16 cout << endl; } 17 }; 18 int main() { 19 MyArray<5> obj; // N yahan 5 hai 20 obj.fill(); 21 obj.display(); // Outputs: 0 10 20 30 40 22 23 return 0; 24 } </pre>	<pre> 0 10 20 30 40 === Code Execut </pre>

. (DOT) operator →

The . (dot) operator in C++ is used to access members (variables and functions) of an object. It's a way to call methods or access properties of a particular instance of a class. Here's a simple explanation and example:

main.cpp	Run	Output
<pre> 1 #include <iostream> 2 using namespace std; 3 class MyClass { 4 public: 5 int value; 6 void display() { 7 cout << "Value: " << value << endl; 8 } 9 }; 10 11 int main() { 12 MyClass obj; // Create an object of MyClass 13 obj.value = 10; // Use . operator to set the value of the // member variable 14 obj.display(); // Use . operator to call the member function 15 16 return 0; 17 } 18 </pre>	Run	Value: 10 === Code Exec

Casting in C++:

Casting ka matlab hai ek type ke variable ko doosre type ke variable mein badalna (convert karna)

C-Style Cast:

- Sabse simple aur purani tareeka.
- Syntax: (type) variable

main.cpp	Run	Output
<pre> 1 #include <iostream> 2 using namespace std; 3 4 int main() { 5 float a = 10.3; // Integer variable 6 int b = (int) a; // C-Style cast to convert int to double 7 8 cout << "Integer value: " << a << endl; // Outputs: Integer 9 cout << "Double value: " << b << endl; // Outputs: Double 10 11 return 0; 12 } 13 </pre>	Run	Integer value: 10.3 Double value: 10 === Code Execution Succes

C++ mein references ka use variables ko ek alias (nickname) देने के लिये होता है। References pointers जैसे ही होते हैं, लेकिन उन्हें initialize करना और use करना ज्यादा simple होता है। Aayein, simple shabdon mein samjhate hain references ko aur ek example dekhte hain:

- **Declaration:** Reference ko declare karte samay usse initialize करना zaroori होता है।
- **Syntax:** type &ref = original_variable;

- **Alias:** Reference kisi original variable ka alias banta hai. Matlab, koi bhi operation jo reference par kiya jata hai, woh original variable par apply hota hai.

main.cpp	Output
<pre> 1 #include <iostream> 2 using namespace std; 3 4 int main() { 5 int a = 10; // Original variable 6 int &ref = a; // Reference 'ref' ko 'a' ka alias banate hain 7 8 cout << "Original value (a): " << a << endl; // Outputs: 10 9 cout << "Reference value (ref): " << ref << endl; // Outputs: 10 10 11 ref = 20; // Reference ko update karte hain 12 13 cout << "Updated original value (a): " << a << endl; // Outputs: 14 20 15 cout << "Updated reference value (ref): " << ref << endl; // 16 Outputs: 20 17 18 return 0; 19 } 20 </pre>	<pre> Original value (a): 10 Reference value (ref): 10 Updated original value (a): 20 Updated reference value (ref): 20 === Code Execution Successful === </pre>

Vectors kya hain?

- Vectors C++ mein ek type ka dynamic array hota hai.
- Vectors ka size badhaya ya ghataaya ja sakta hai jab zaroorat ho.

main.cpp	Output
<pre> 1 #include <iostream> 2 #include <vector> // Vector library ko include karte hain 3 using namespace std; 4 5 int main() { 6 vector<int> vec; // Ek integer type ka vector declare karte hain 7 8 // Vector mein elements add karte hain 9 vec.push_back(10); 10 vec.push_back(20); 11 vec.push_back(30); 12 13 // Elements ko access karte hain using index 14 cout << "Element at index 0: " << vec[0] << endl; // Outputs: 10 15 cout << "Element at index 1: " << vec[1] << endl; // Outputs: 20 16 cout << "Element at index 2: " << vec[2] << endl; // Outputs: 30 17 18 return 0; 19 } 20 </pre>	<pre> Element at index 0: 10 Element at index 1: 20 Element at index 2: 30 === Code Execution Successful === </pre>

Find size →

main.cpp	Output
<pre>1 #include <iostream> 2 #include <vector> // Vector library ko include karte hain 3 using namespace std; 4 5 int main() { 6 vector<int> vec; // Ek integer type ka vector declare karte hain 7 8 // Vector mein elements add karte hain 9 vec.push_back(10); 10 vec.push_back(20); 11 vec.push_back(30); 12 13 // Elements ko access karte hain using index 14 cout << "Element at index 0: " << vec[0] << endl; // Outputs: 10 15 cout << "Element at index 1: " << vec[1] << endl; // Outputs: 20 16 cout << "Element at index 2: " << vec[2] << endl; // Outputs: 30 17 18 cout << "Size of vector: " << vec.size() << endl; // Outputs: 3 19 20 return 0; 21 } 22</pre>	<pre>Element at index 0: 10 Element at index 1: 20 Element at index 2: 30 Size of vector: 3 === Code Execution Success</pre>

Type Def :

Agar aap directly data types use karte hain, toh functionality par koi farak nahi padta. Lekin typedef ka fayda ye hai ki aapke code ko zyada readable aur maintainable banata hai, khas kar jab complex data types ya long declarations hote hain.

Without typedef :

main.cpp	Output
<pre>1 #include <iostream> 2 using namespace std; 3 4 struct Student { 5 string name; 6 int age; 7 }; 8 9 int main() { 10 struct Student s1; 11 s1.name = "John"; 12 s1.age = 20; 13 14 cout << "Name: " << s1.name << ", Age: " << s1.age << endl; 15 16 return 0; 17 } 18</pre>	<pre>Name: John, Age: 20 === Code Execution Success</pre>

With :

main.cpp	Output
<pre>1 #include <iostream> 2 using namespace std; 3 4 typedef struct { 5 string name; 6 int age; 7 } Student; 8 9 int main() { 10 Student s1; 11 s1.name = "John"; 12 s1.age = 20; 13 14 cout << "Name: " << s1.name << ", Age: " << s1.age << endl; 15 16 return 0; 17 } 18</pre>	Name: John, Age: 20 === Code Execution Su

Advantages:

- **Readability:** Code zyada readable aur understandable ho jata hai.
- **Maintenance:** Code ko maintain karna easy ho jata hai.
- **Clarity:** Complex types ko simplify karke clarity badhata hai

::

Recursive function ek aisi function hoti hai jo apne aap ko call karti hai. Yeh useful hoti hai jab aapko kisi problem ko smaller subproblems mein tod ke solve karna hota hai. Recursive functions ko use karte waqt base case aur recursive case ka dhyaan rakhna padta hai.

Key Components of Recursive Function:

1. **Base Case:** Yeh condition hoti hai jo recursion ko end karti hai. Without base case, recursive function infinite loop mein fas sakti hai.
2. **Recursive Case:** Yeh part function ko call karta hai apne aap ko with different arguments.

main.cpp	Output
<pre>1 #include <iostream> 2 using namespace std; 3 4 // Recursive function to calculate factorial 5 int factorial(int n) { 6 if (n == 0) { 7 return 1; // Base case: factorial of 0 is 1 8 } else { 9 return n * factorial(n - 1); // Recursive case 10 } 11 } 12 int main() { 13 int number = 5; 14 cout << "Factorial of " << number << " is " << factorial(number) 15 << endl; // Outputs: 120 16 } 17</pre>	Factorial of 5 is 120 === Code Execution Suc

- **Base Case:** `if (n == 0) return 1;` yeh condition recursion ko end karti hai.
- **Recursive Case:** `return n * factorial(n - 1);` yeh part function ko call karta hai with `n-1`.

Example with Steps:

For `factorial(5)`:

1. `factorial(5)` calls `factorial(4)`
2. `factorial(4)` calls `factorial(3)`
3. `factorial(3)` calls `factorial(2)`
4. `factorial(2)` calls `factorial(1)`
5. `factorial(1)` calls `factorial(0)`
6. `factorial(0)` returns 1
7. Then the functions return values step by step:
 - `factorial(1)` returns $1 * 1 = 1$
 - `factorial(2)` returns $2 * 1 = 2$
 - `factorial(3)` returns $3 * 2 = 6$
 - `factorial(4)` returns $4 * 6 = 24$
 - `factorial(5)` returns $5 * 24 = 120$

ARGUMENT AND WITHOUT ARGUMENT IN C++

main.cpp	Output
<pre>1 #include <iostream> 2 using namespace std; 3 4 // Function with arguments 5 void greet(string name) { 6 cout << "Hello, " << name << "!" << endl; 7 } 8 9 int main() { 10 greet("Alice"); // Outputs: Hello, Alice! 11 greet("Bob"); // Outputs: Hello, Bob! 12 return 0; 13 } 14</pre>	<pre>Hello, Alice! Hello, Bob! === Code Execut</pre>

- `greet(string name)` ek function hai jo ek argument name leta hai.
- `greet("Alice")` aur `greet("Bob")` function calls hain jo name ko "Alice" aur "Bob" set karte hain.

Without Arguments →

main.cpp	Output
<pre>1 #include <iostream> 2 using namespace std; 3 4 // Function without arguments 5 void greet() { 6 cout << "Hello, World!" << endl; 7 } 8 9 int main() { 10 greet(); // Outputs: Hello, World! 11 return 0; 12 } 13</pre>	<pre>Hello, World! === Code Executi</pre>